

BRNO UNIVERSITY OF TECHNOLOGY
VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY
FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF COMPUTER GRAPHICS AND MULTIMEDIA
ÚSTAV POČÍTAČOVÉ GRAFIKY A MULTIMÉDIÍ

ADVANCED INJURY SIMULATION OF GAME CHARACTERS

POKROČILÁ SIMULACE ZRANĚNÍ HERNÍCH POSTAV

BACHELOR'S THESIS
BAKALÁŘSKÁ PRÁCE

AUTHOR **ADAM HELEŠIC**
AUTOR PRÁCE

SUPERVISOR **Ing. TOMÁŠ CHLUBNA, Ph.D.**
VEDOUCÍ PRÁCE

BRNO 2026

BRNO UNIVERSITY OF TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY

Bachelor's Thesis Assignment

172352

Institut:	Department of Computer Graphics and Multimedia (DCGM)
Student:	**Helešic Adam**
Programme:	Information Technology
Title:	**Advanced Injury Simulation of Game Characters**
Category:	Computer Graphics
Academic year:	2025/26

Assignment:

1. Familiarize yourself with game development in the Godot engine.
2. Study the issue of game character animation and damage during combat.
3. Research existing and possible solutions to the given problem.
4. Design a user interface and the final functionality of an application that demonstrates advanced character reactions to injuries.
5. Implement the application and demonstrate its use with different types of data.
6. Perform user or performance testing of the results.
7. Document and publish the final application.
8. Create a video representing the results of your work.

Literature:

- Adams, Ernest, and Joris Dormans. Game mechanics: advanced game design. New Riders, 2012. ISBN 0321820274, 9780321820273.
- Geijtenbeek, T., Vasilescu, D., Egges, A. Injury Assessment for Physics-Based Characters. In: Allbeck, J.M., Faloutsos, P. (eds) Motion in Games. MIG 2011. Lecture Notes in Computer Science, vol 7060. Springer, Berlin, Heidelberg. 2011. DOI: 10.1007/978-3-642-25090-3_7.
- Francis Laclé, Nicolas Pronost. Real-Time Musculoskeletal Model for Injury Simulation on 3D Human Characters. Motion In Games Conference (MIG), Nov 2012, Rennes, France.
- Aristidou, A., Lasenby, J., Chrysanthou, Y. and Shamir, A. Inverse Kinematics Techniques in Computer Graphics: A Survey. Computer Graphics Forum, 37: 35-58. 2018. DOI: 10.1111/cgf.13310.

Requirements for the semestral defence:
Items 1-4, experiments leading to item 5.

Detailed formal requirements can be found at https://www.fit.vut.cz/study/theses/

Supervisor:	**Chlubna Tomáš, Ing., Ph.D.**
Head of Department:	Černocký Jan, prof. Dr. Ing.
Beginning of work:	1.11.2025
Submission deadline:	13.5.2026
Approval date:	7.11.2025

Abstract

Do tohoto odstavce bude zapsán výtah (abstrakt) práce v anglickém jazyce.

Abstrakt

Do tohoto odstavce bude zapsán výtah (abstrakt) práce v českém (slovenském) jazyce.

Keywords

Sem budou zapsána jednotlivá klíčová slova v anglickém jazyce, oddělená čárkami.

Klíčová slova

Sem budou zapsána jednotlivá klíčová slova v českém (slovenském) jazyce, oddělená čárkami.

Reference

HELEŠIC, Adam. *Advanced Injury Simulation of Game Characters.* Brno, 2026. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor Ing. Tomáš Chlubna, Ph.D.

Advanced Injury Simulation of Game Characters

Declaration

Prohlašuji, že jsem tuto bakalářskou práci vypracoval samostatně pod vedením pana X... Další informace mi poskytli... Uvedl jsem všechny literární prameny, publikace a další zdroje, ze kterých jsem čerpal. Pro jazykovou revizi technické zprávy jsem použil(a) nástroj ChatGPT. Při tvorbě odevzdaného programového kódu jsem využíval nástroj Github Copilot. [konkretizujte a upravte podle skutečnosti].

.....
Adam Helešic
January 13, 2026

Acknowledgements

V této sekci je možno uvést poděkování vedoucímu práce a těm, kteří poskytli odbornou pomoc (externí zadavatel, konzultant apod.).

Contents

1 Introduction **2**

2 Game Character Injury Systems and Animation **3**
2.1 Game Design 3
2.2 Case Studies 4
2.3 Locomotion 6
2.4 Enemy AI 9

3 Injury System Design **12**
3.1 Enemy Design 12
3.2 Enemy AI Based on Body Parts 13
3.3 UI Mockup 13

Bibliography **14**

Chapter 1

Introduction

The goal of this thesis is the design and implementation of a dynamic injury system of game characters in the Godot Engine. The term *character* here describes game characters in general, and is not intended to be limited to anthropomorphs. This demo is not player-centric, but rather aims to implement a standalone injury system and its effects on the creature's behavior and animation.

Chapter 2 explores existing injury systems from a design perspective and explains the basics of the mechanisms that are used in the implementation of the final product, which include inverse kinematics (for character animation) and AI patterns (specifically the finite state machine and the behavior tree).

Chapter 3 details the design of the game demo. It explains the design of the enemy model, the subsystems necessary for implementing a cohesive enemy with a specialized injury system, and showcases mockups of the in-engine tools' user interface.

Chapter 2

Game Character Injury Systems and Animation

Combat and injury have been a part of human history even since prehistoric times. Physical conflict accompanies many stories, large-scale historical events, and even modern times. It is then no wonder that games have used this theme in a myriad of ways. From mythical, heroic combat with swords and magic to competetive shooters, countless games had to contend with designing and implementing combat and injury systems.

Each game has its budget and own set of priorities based on their design. Sometimes, developers may decide to prioritize other parts of the game like visuals, story, and voice acting over the implementation of complex combat systems, opting for greater abstraction. The easiest systems are the ones that model characters as a singular entity with no subparts and a number representing hit points that determines its binary state – alive or dead. However, other designs call for a more complex and varied experience.

This chapter delves into the basics of game design and examines how other games have developed such systems. Next, it looks into the mechanisms constituting an advanced injury system of game characters – systems of simulating harm and injury, locomotion, and enemy artificial intelligence (AI) that reflects the injuries in its decision making.

2.1 Game Design

Game design is the act of deciding what a game should be. Decisions about rules, look and feel, timing, pacing, risk-taking, rewards, punishments, and everything else the player experiences is the responsibility of the game designer [6]. Schell proposes a very player-centric definition – *anything* that the player experiences in the game is part of the game design. It is a fair, but broad explanation. A lower level approach may be more useful in practice when creating a new design.

Adams and Dormans argue that games are built on a set of rules that together define the play experience. There are a few principles to designing rulesets. Firstly, a game should have an extent of unpredictability (but not be too unpredictable nor a predetermined sequence of events). Secondly, the players' skill, strategic decisions, and knowledge of the games should make a difference. The longer the game, the more should skill contribute to victory. Thirdly, complex gameplay facilitated by many interactive parts and interweaving rules adds another layer of unpredictability and creates an opportunity for player skill and game knowledge to matter. The player is incentivized to learn to read and foresee the game

state and understand the strategic complexities to make judgement calls based on these observations. Most games combine these three sources of creating engagement – chance, player choices, and complex rules [1].

The game design in this thesis aims to create a more complex combat experience with less abstractions, where the player can make meaningful choices about where to attack an enemy to cripple its capabilities based on the body part harm. This is the lens through which later sections examine existing implementations in games. While assessing, designing, and implementing these features, the goal is also to develop in-engine tools that would make creating such enemies more streamlined in Godot.

Simulating Harm and Injury

The implementation of injury depends on where the design lands on the spectrum between realism and abstraction for the sake of gameplay. One avenue would be to simulate the game character as a physics-based entity with detailed models of musculature. Injury would be detected precisely where a collision with a weapon happens, and articulated movement that uses the muscles in those parts disabled. However, the goal of this thesis is not to create a perfectly realistic body simulation, but to provide a framework and streamline creating enemies with targetable body parts that serve as prerequisites for enemy abilities, and making it publically available to Godot users.

2.2 Case Studies

This section examines the systems within games that implement a body part injury system and their approach to the design.

In Horizon Zero Dawn and Horizon Forbidden West, components and armor plates can be removed with high tear damage weapons, which deals little to no damage, but reveals weak points or disables attacks[1].

In Monster Hunter Wilds, removing body parts is a core part of the experience, gathering them as loot or removing the monsters' dangerous abilities. Different weapon types (slashing, blunt, ranged) deal increased damage to specific enemies' body parts. For example, slashing weapons deal extra damage to tails, while blunt weapons can stun, making the enemy roll on the ground for a period of time[2].

Dead Space has many enemies with weak spots, different for each enemy type. Attacking those parts can limit an enemy's attack or movement. Some enemies can have armor that protects their weak points, forcing the player to focus on another weak point or to change their position to access weak points hidden on the enemy's back. Enemies can also have weak points that are dangerous to the player, such as explosive sacks damaging the player when destroyed in close proximity, forcing another angle of strategy to the weak point system. The important aspect of this combat system is in how it enforces different strategies that the player needs to apply based on the enemy type's weak point effects[3].

Fallout offers a slow-motion targeting system that allows players to target specific body parts. Each body part has its own chance of being hit, modified by distance and cover.

[1] https://www.polygon.com/horizon-zero-dawn-guide-walkthrough/21357205/machines-fight-rockbreaker-behemoth-thunderjaw-stormbird-deathbringer-tallneck-corrupted-cauldron/

[2] https://www.polygon.com/monster-hunter-wilds-guide/528353/monster-tails-fangs-claws-horns-how-to-get-sever-cut-break/

[3] https://www.dualshockers.com/dead-space-remake-necromorph-weakness/



Figure 2.1: First row: Dead Space's necromorph enemies with harmable limbs highlighted red[7]. Second row, left to right: Fallout's V.A.T.S.[8]and Phoenix Point's free aim targeting[9].

The number of attacks a player can make is dependent on the player's action points and the weapon type. Body parts have a status that's separate from the target's overall health. A status bar can be worn down until empty, at which point the limb becomes crippled. Crippled limbs have effects like hindered movement, removing a robot's weaponized arm, or shooting an enemy's head (which in most cases kills them outright)[4].

In Xenoblade Chronicles X, the player can target appendages which have their own health pool, separate from the overall one. Destroying appendages disables certain attacks and lowers the enemy's defenses[5].

Phoenix Point's "free aim" combat allows the player to hit specific body parts based on projectile collisions. Each body part has their independed values (like health) and features. Disabling parts has various effects, such as hindering mobility, disabling the enemy's main weapon, or triggering a bleeding effect. Each part also has a different armor value which reduces damage[6].

[4] https://www.videogamer.com/guides/a-beginners-guide-to-combat-in-fallout-4/

[5] https://deltiasgaming.com/xenoblade-chronicles-x-combat-guide/

[6] https://www.rockpapershotgun.com/phoenix-point-free-aim-and-part-damage-guide-how-to-use-the-free-aiming-system-for-more-precise-shots

[7] Image taken from https://vulkk.com/2023/02/03/dead-space-remake-combat-tips-how-to-defeat-each-type-of-necromorph/.

[8] Image taken from https://www.thegamer.com/fallout-3-vats-bethesda-nearly-cut/.

[9] Image taken from https://www.rockpapershotgun.com/phoenix-point-free-aim-and-part-damage-guide-how-to-use-the-free-aiming-system-for-more-precise-shots.

Summary

Body parts most often have a feature bound to them – an ability (like a weapon attack), protection (armor that protects weak points or targetable body parts from being targeted), or mobility. Attacking such body parts hinders those features, e.g., slowing an enemy down or disabling an attack. Additionally, body parts may serve as weak points, granting the player an advantage when being hit in a particular way (like stunning enemies by hitting their heads with a blunt weapon in Monster Hunter Wilds).

There are also variable values that modify the way limbs take damage, such as a hit chance (Fallout) or armor that reduces damage taken (Phoenix Point).

The designs also aim for enemy variance that enforces different strategies to be employed by the player (like in Dead Space).

Game designs should also be mindful of not overwhelming the player. This can be solved by making combat systems more diegetic, meaning embedding mechanics into to the "narrative world" (the game world as perceived by the game characters). For example, whichever body part the player shoots, that part takes damage, as opposed to the player choosing from several invasive UI popups.

2.3 Locomotion

Locomotion is the problem of making game characters move. Often, it is not a single system, but an intertwining of AI, gameplay, physics, and animation systems [4].

This thesis concerns itself mainly with enemy AI and animation. Inverse kinematics is used to make simulated movement look more natural, allowing for end effectors, such as feet, to be placed dynamically based on things like raycast collisions with the terrain mesh (see adjusted foot height in Figure 2.2). Inverse kinematics can also be used for attaching end effectors to a point on the ground, correctly aiming for a target, etc. This can prevent awkward inconsistencies like sliding walk animations or animations when reaching for an object is offset from the actual target due to a static animation.

The injury system has to be dynamic, adapting to whichever injuries are applied to the character's body. Hence procedural animation fits this goal of adaptibility well. Rather than traditional animation with preset model movements, procedural animation can account for and adapt to injuries on the run, and make for a more realistic and immersive interaction with the injured character.

Inverse Kinematics

Inverse kinematics is *the rotational and translational motion of points, bodies (objects) and systems of bodies (groups of objects) without consideration of what causes the motion or any reference to mass, force or torque* [2]. In short, it is about moving and rotating a chain of links to reach a specific point.

It had originally been used in robotics for moving arms to targets, but has also expanded into animation in computer graphics. In the latter field, it is very useful for animating *articulated figures* (figures with a specific, solid form like humans, animals, and generally creatures with limbs). These models are often complicated, and require efficient calculation of complex and multi-link models that can have specified constraints (e.g., a maximum angle that a link can turn by). The multi-link models of articulated figures in inverse kinematics



Figure 2.2: Dynamic foot placement using inverse kinematics (The Witcher 3).

are calculated using the positions of the model's joints, known as *end effectors* (often the end joints), their potential constraints, and the target positions to be reached [2].

A *model* is a series of *links* connected by *joints*. Joints are the main component allowing movement – their positions determine the positions and orientations of the links. Links are the "connective tissue" between joints and are defined by their length, shape, volume, and mass. A *posture* is the skeletal configuration of the figure and must most often satisfy constraints to adhere to realistic movement, such as the torque limitations of each joint or the links not intersecting with other models in their path. A skeletal configuration is usually separated into *chains*, standalone parts attached to a static point on the model (e.g., the limbs of animals and anthropomorphs). The chains usually work in a hierarchy, and the movement of one joint affects the movement of the ones before it [2]. An example model can be seen in Figure 2.3. The way the movements of joints affect each other is illustrated in Figure 2.4. The target is reachable in (b) and (c), while (d) showcases the attempt of the chain to reach an unreachable target. The specific way these links bend depends on the concrete inverse kinematics solver implementation.

Usage of Inverse Kinematics

Forward kinematics (traditional animation where known, predefined transformations are applied to the model) can fall short in situations where dynamic adjustment of the target locations may be needed, such as characters standing on slopes or an appandage reaching for a target that can be a variable distance or height away. The bending of the links (such as limbs) is dynamically calculated, making for more natural-looking and adaptive movement, which is a desirable trait for our dynamic body part injury system that needs to account for the injuries made.

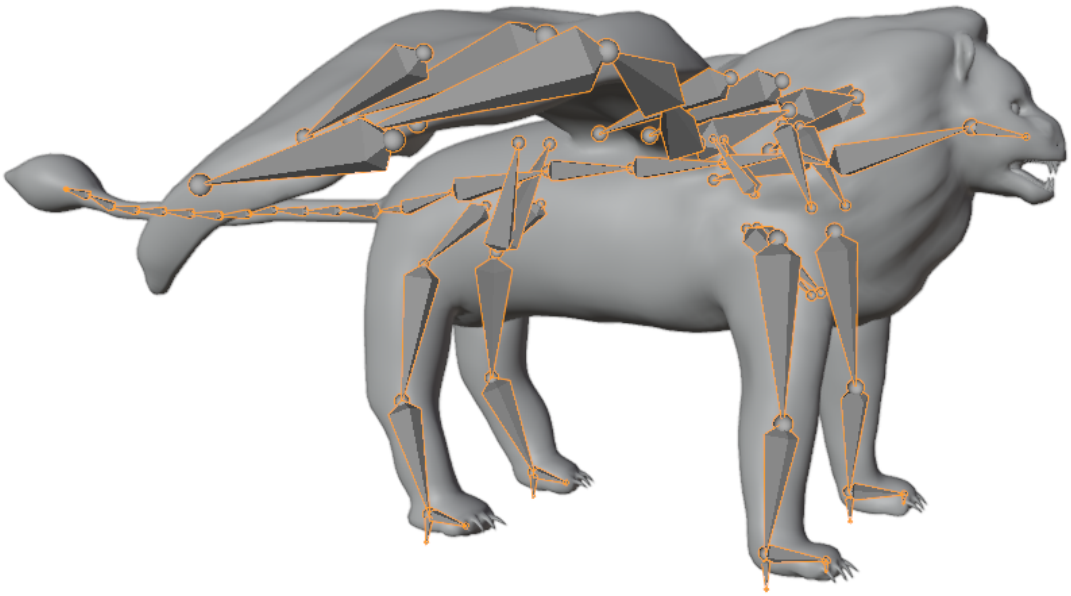


Figure 2.3: An inverse kinematics *model* – a skeleton rig of the game character consisting of *chains* like the limbs, the tail, or the wings.

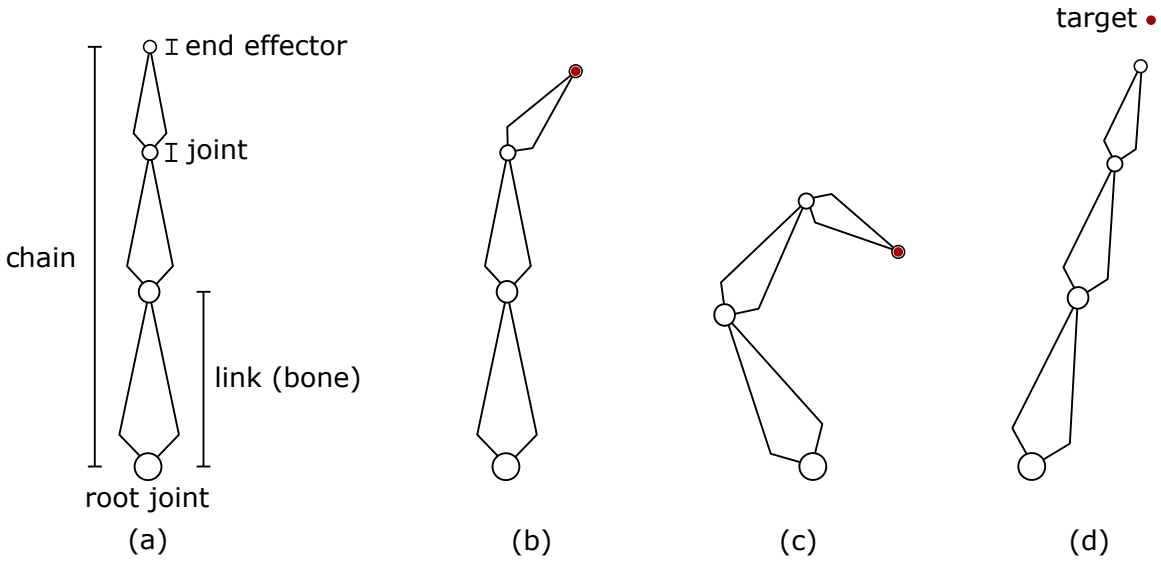


Figure 2.4: An inverse kinematics chain description and examples of the chain bending so that the end effector reaches the red dot target, showing two reachable and one unreachable target point positions.

2.4 Enemy AI

The enemy AI requires the solving of two main problems – deciding and executing behavior, and pathfinding.

2.4.1 Finite State Machine

One of the possible implementations of enemy AI is a finite state machine (FSM). FSMs are an architectural structure that can encapsulate enemy behavior into separate states, such as *Patrol* or *Attack*. A central directing *state machine* controls which state is active and allows for the state to be changed. The *state* itself has the encapsulated behavior logic (e.g., a patroling enemy walking between two points) and the transition logic like methods to be executed when entering or exiting a state [5]. A simple FSM implementation is illustrated in Figure 2.5.

Finite state machines are a quick and easy implementation of enemy AI where the behavior can be separated into states and where there are clear transitions in each state. It can get convoluted if the transition requirements are complex – in the most extreme cases, it would be FSMs where each state can transition to any other state.

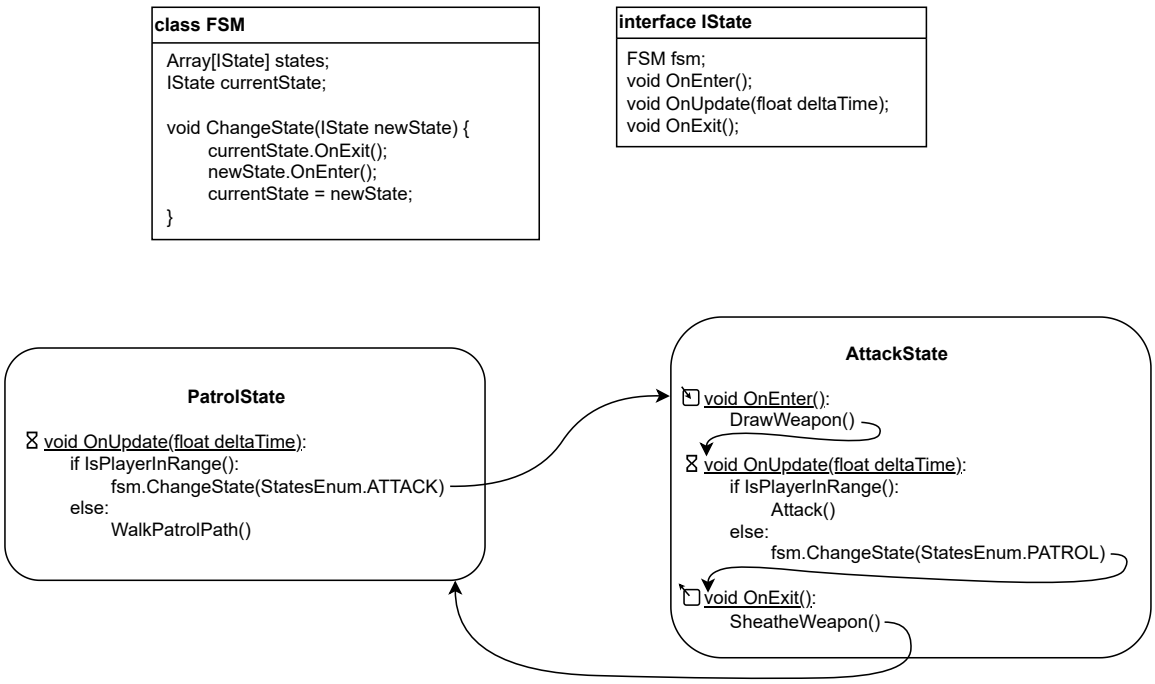


Figure 2.5: An example of a finite state machine implementing a simple enemy AI.

2.4.2 Behavior Tree

Another option for enemy AI implementation is the behavior tree (BT), a way to structure decision making logic into a more streamlined form. BTs follow the traditional node-based architecture of trees, but contain special node types that serve the node traversal and behavior logic. A sample BT and its traversal is illustrated in Figure 2.6.

The node type and implementation descriptions from this section are taken from [3]. The most commonly used nodes are as follows:

- *Behavior* is an abstract interface that can be activated, run, and deactivated, and passes back a *return status*. The concrete implementations of this interface are *actions* (accessing information and making changes to the world) and *conditions* (a primary way of checking information from the world).
- *Sequences* can be thought of as sequential `AND` gates – each of their children nodes are executed in order one after another, until all have executed successfully or until one of the children fails. Not only is this useful for chaining behavior, but also for setting up conditions for whether to execute this branch at all – if a condition node is added as its first child, it must return `true` for the sequence to continue executing.
- *Selectors* can be thought of as sequential `OR` gates or fallbacks. It executes its children in order until one returns `SUCCESS` or `RUNNING`. If a child fails, the next one is its "fallback" behavior.
- *Parallels* execute all of their child nodes simultaneously. It is important to clearly define these nodes' return states (e.g., returns `SUCCESS` only if all of the child nodes succeed). When exiting, the termination of all child nodes must also be handled cleanly (although in some cases, like animations and sound effects, a delay in the child's terminatation to after it has finished its action may be desired).

Return statuses are crucial for tree traversal. Most often, behavior nodes return whether they completed their run as expected or not, or that they're still running – `SUCCESS`, `RUNNING`, or `FAILURE`. However, BTs can also leverage the return status for more efficient implementations, such as the `SUSPENDED` status code in event-driven BTs.

This basic BT implementation can then be extended by more specialized nodes.

- *Decorators* (named after the design pattern) allows behaviors to be wrapped in other behaviors, such as repeating a child's behavior n times.
- *Filters* will not execute its children under specific conditions (such as an action cooldown or a target being out of range of an action). Filters can be easily created by attaching a condition to the start of a sequence, as described above. Advanced filters can be created by combining sequence (`AND`), selector (`OR`), and condition nodes.
- *Monitors* serve for continously checking if assumptions for an active behavior are valid, and signal that the whole subtree should be exited when they become invalid. It runs in parallel with other nodes, usually a single sibling that is a behavior subtree that needs some continuous state checking (to ensure it makes sense to keep running it).
- *Active selectors* actively recheck their decisions regularly, allowing them to retry higher priority branches (e.g., interrupting a patrol with a search behavior if the player is spotted).

It is useful to have a `BehaviorTree` object that is built by a `BehaviorTreeBuilder`. This will serve as the central point for storing and updating the tree. The BT has an `update` method which performs the traversal of the tree and is the entry point of the BT. It is called whenever an update is needed, though it need not be every game frame – every other frame or 5 frames per second can suffice (to lighten or spread out the processing load). For a more efficient approach, an event-based node traversal implementation is possible where a central scheduler manages the traversal using the observer pattern.

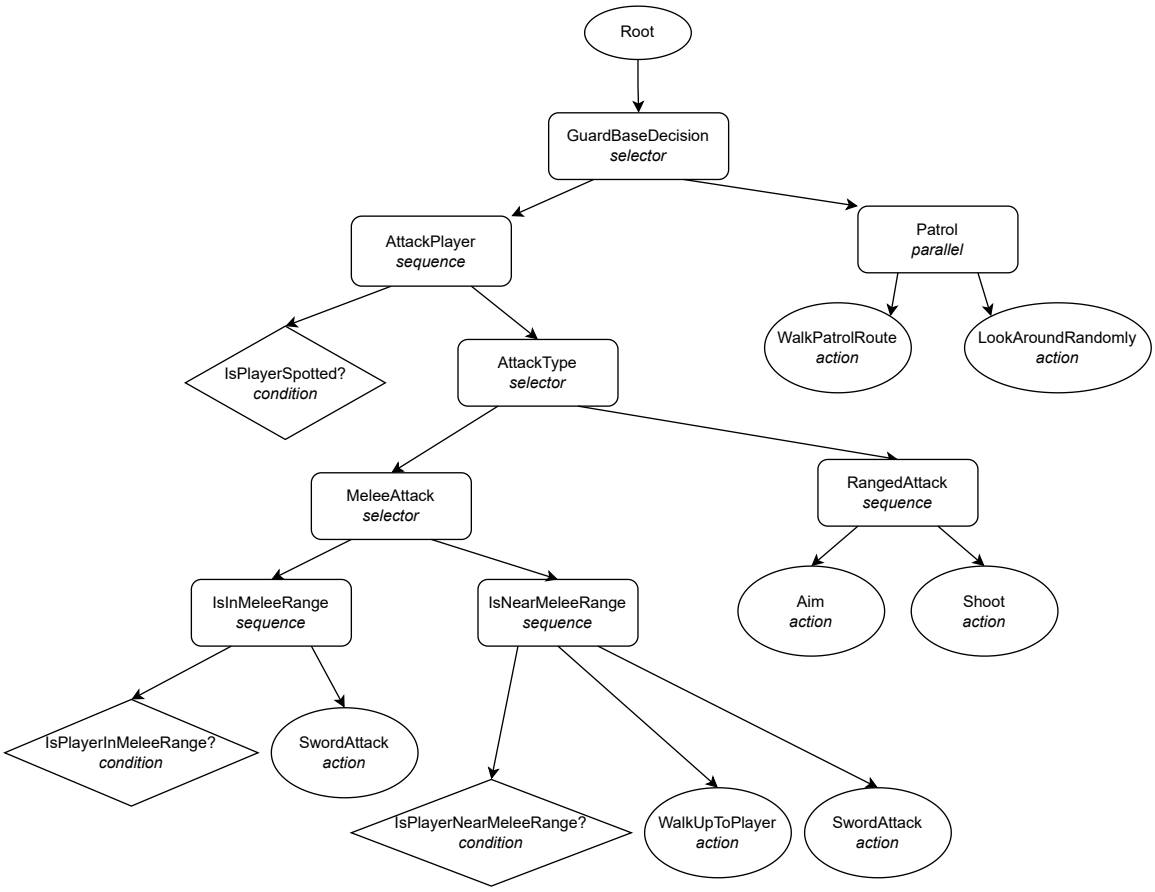


Figure 2.6: An example of an enemy AI behavior tree. A possible traversal in a situation where the player is not seen would go as described: The `GuardBaseDecision` seletor enters `AttackPlayer` (its first child). As a sequence, this node has to run all of its children in order to succeed. The `IsPlayerSpotted?` condition is false, and thus the whole sequence returns the `FAILURE` return state. The `GuardBaseDecision` selector then enters its next child (the fallback), which is a parallel node with two action children. The `WalkPatrolRoute` and the `LookAroundRandomly` actions start simultaneously executing.

Chapter 3

Injury System Design

Navrhněte uživatelské rozhraní a výslednou funkcionalitu aplikace, která demonstruje pokročilé reakce herní postavy na zranění.

The player should be able to make meaningful choices that have a real impact on what happens next [6]. For example, a player that prefers melee combat might target an enemy's wings to prevent it from flying and getting out of melee range.

The game should combine the three sources of unpredictability mentioned in Chapter 2 to make it more engaging to players. The element of chance comes from the enemy's AI which makes randomized decisions based on a decision-making pattern like a behavior tree. The player chooses which body parts to attack so that they can disable the enemy's abilities that require those parts to be triggered. The complexity comes from the way body parts serve as prerequisites for the enemy's abilities, such as two wings being necessary for flight, at least three limbs for walking, or a stinger tail for poison attacks, and the way they enable and limit what actions it has available in its behavior.

Depending on the implementation of the player controller, there are even more elements of player choice and rules complexity. Such cases could include the player maneuvering around the virtual space to avoid or close in on an enemy, or any of the player's special abilities and attacks with various cooldowns and effects. However, the player controller is a complex issue of its own. It is not covered by this thesis, but the player's experience is a crucial aspect of games and should be accounted for when discussing game design.

Design justified by design principles (books).

There are many possible implementations of a body part injury system. HP-based; body-part based; idk

probably going with a joint HP pool where each body part is some fraction of that pool

losing a body part needs to have a gameplay effect (quote games)

3.1 Enemy Design

4 limbs, wings, tail – targetable body parts

a skeleton rig that allows its mesh to be deformed to reach a target point using IK

3.2 Enemy AI Based on Body Parts

3.2.1 Behavior Execution

what pattern to use when simulating the behavior? something that will change states like “Fly Up”, “Descend”, “Rake with Claws”, but what will take care of the actual behavior? And what is the central brain that manages the state?

chosen Behavior Tree

3.2.2 Behavior Limiting

patterns to check for available actions? how to limit effectively what the Behavior Tree (or the chosen way to implement behavior) has access to? so that when the random value in the weighted network is generated, so that it doesn't try to execute a branch that cannot be executed due to missing links

The behavior also needs to be limited
deciding on actions and movement;
limiting options based on harm;
procedural animation of actions

3.3 UI Mockup

Godot plugin design and mockups

Bibliography

[1] Adams, E. and Dormans, J. *Game Mechanics: Advanced Game Design*. New Riders, 2012. Voices That Matter Series. ISBN 9780321820273.

[2] Aristidou, A.; Lasenby, J.; Chrysanthou, Y. and Shamir, A. Inverse kinematics techniques in computer graphics: A survey. *Comput. Graph. Forum* online. Wiley, September 2018, vol. 37, no. 6, p. 35–58. ISSN 0167-7055. Available at: https://doi.org/10.1111/cgf.13310. [cit. 2026-01-08].

[3] Champandard, A. J. and Dunstan, P. The Behavior Tree Starter Kit. In: Rabin, S., ed. *Game AI Pro* online. 1st ed. CRC Press, September 2013. ISBN 9781466565975. Available at: https://www.gameaipro.com/GameAIPro/GameAIPro_Chapter06_The_Behavior_Tree_Starter_Kit.pdf.

[4] Ciupinski, J. Animation-Driven Locomotion with Locomotion Planning. In: Rabin, S., ed. *Game AI Pro* online. 1st ed. CRC Press, September 2013. ISBN 9781466565975. Available at: http://www.gameaipro.com/GameAIPro/GameAIPro_Chapter25_Animation-Driven_Locomotion_with_Locomotion_Planning.pdf.

[5] Graham, D. A Reusable, Light-Weight Finite-State Machine. In: Rabin, S., ed. *Game AI Pro 3* online. 1st ed. CRC Press, June 2017. ISBN 9781498742580. Available at: https://www.gameaipro.com/GameAIPro3/GameAIPro3_Chapter12_A_Reusable_Light-Weight_Finite-State_Machine.pdf.

[6] Schell, J. *The Art of Game Design: A Book of Lenses*. CRC Press, Taylor & Francis Group, 2019. Computer graphics / Game design. ISBN 9781138632097.